

Uber Rides Data Analytics Project

Author: Ayush Kumar
Institution: Chandigarh University
Location: Patna, Bihar, India
GitHub: <https://github.com/ayush-9955/Uber-Rides-.git>
Last Updated: March 2026

Project Overview

This repository contains a comprehensive end-to-end data analytics project analyzing Uber rides data. The project demonstrates proficiency in data cleaning, exploratory data analysis (EDA), SQL querying, Python scripting, and Power BI dashboard creation. Using real Uber trip datasets, the analysis uncovers ride patterns, demand forecasting, peak hours, geographic hotspots, and revenue optimization insights—ideal for data analyst portfolio demonstrations and technical interviews.

The project workflow follows industry-standard ETL (Extract, Transform, Load) processes: raw data import via Excel, transformation and querying through SQL, advanced analytics using Python libraries, and interactive visualization through Power BI dashboards with DAX measures and KPIs.

Repository Structure

The repository is organized into five main folders representing different stages of the analytics pipeline:

Folder	Description	Key Contents
Excel	Raw datasets and initial data cleaning	CSV exports, pivot tables, data validation
Power BI	Interactive dashboards with 16 KPIs	.pbix files, DAX measures, 12+ charts
SQL	Database queries for aggregation and joins	Ride volume analysis, fare calculations, window functions
Python	EDA, feature engineering, and modeling	Jupyter notebooks, pandas/numpy scripts, visualizations
Ydata_Profiling	Automated data profiling reports	HTML reports, statistical summaries

Table 1: Repository folder structure and contents
Additional files include desktop.ini configuration and supporting documentation.

Technical Stack

- **Programming Languages:** Python 3.x, SQL
 - **Python Libraries:** pandas, numpy, matplotlib, seaborn, scikit-learn
 - **Database:** SQL Workbench, MySQL/PostgreSQL
 - **Visualization:** Power BI Desktop (DAX, star schema, slicers)
 - **Data Profiling:** ydata-profiling (pandas-profiling)
 - **Version Control:** Git, GitHub
 - **Data Format:** CSV, Excel (.xlsx), JSON
-

Key Features and Insights

Temporal Analysis

- Peak ride hours identified during evening rush (5-7 PM) with 35% higher trip volume
- March 2016 shows +584% month-over-month revenue spike attributed to longer-distance trips
- Weekend vs weekday patterns reveal distinct demand curves for trip planning optimization
- Year-over-year growth calculated using DAX measures in Power BI dashboards

Geospatial Intelligence

- Heatmaps identify high-demand urban zones using Python geospatial libraries
- Route optimization analysis for efficiency gains in metropolitan areas
- Location-based filtering in Power BI enables stakeholder presentations by region
- Distance and duration correlations mapped for fare proxy calculations

Performance Metrics

- Average speed: 45 mph for business trips during morning hours
- Total trips analyzed: 1,247 rides across 2016 dataset
- Total miles covered: 15,000+ miles with duration analysis
- Revenue proxy formula: $\text{Revenue} = \text{Miles} \times \text{Duration Factor}$

Business Insights

1. **Pricing Optimization:** Dynamic pricing recommendations during peak hours increase revenue potential by 20-30%
 2. **Demand Forecasting:** Predictive models using time-series analysis forecast ride volume for resource allocation
 3. **Route Efficiency:** High-mileage routes identified for driver incentive programs
 4. **Customer Segmentation:** Business vs personal trip patterns enable targeted marketing strategies
 5. **Cancellation Analysis:** SQL queries identify cancellation hotspots for service improvement
-

Python Code Examples

Data Loading and Initial EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Load Uber dataset

```
df = pd.read_csv('uber_rides.csv')
```

Basic statistics

```
print(df.describe())
print(df.info())
```

Create revenue proxy

```
df['fare_proxy'] = df['miles'] * df['duration'] * 0.5
```

Identify peak hour

```
peak_hour = df.groupby('hour')['trips'].sum().idxmax()
print(f"Peak ride hour: {peak_hour}:00")
```

Geospatial Heatmap Visualization

```
import seaborn as sns
```

Group by location coordinates

```
location_density = df.groupby(['lat', 'lon']).size().reset_index(name='ride_count')
```

Create heatmap

```
plt.figure(figsize=(12, 8))
sns.scatterplot(data=location_density, x='lon', y='lat', size='ride_count',
hue='ride_count', palette='YlOrRd', sizes=(20, 500))
plt.title('Uber Ride Density Heatmap')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```

SQL Query Examples

Total Rides and Revenue by Month

```
SELECT
MONTH(ride_date) AS month,
COUNT(trip_id) AS total_trips,
SUM(miles * duration) AS revenue_proxy,
AVG(miles) AS avg_distance
FROM uber_rides
GROUP BY MONTH(ride_date)
ORDER BY month;
```

Peak Hour Analysis with Window Functions

```
SELECT
HOUR(pickup_time) AS hour,
COUNT(
) AS rides, RANK() OVER (ORDER BY COUNT() DESC) AS hour_rank
FROM uber_rides
GROUP BY HOUR(pickup_time)
ORDER BY rides DESC
LIMIT 5;
```

Power BI Dashboard Components

1. **KPI Cards:** Total trips, total miles, average duration, revenue proxy

2. **Time Series Charts:** Monthly/daily ride trends with MoM growth percentages
3. **Geographic Maps:** Ride distribution by pickup and dropoff locations
4. **Bar Charts:** Top 10 routes, hourly distribution, day-of-week patterns
5. **Slicers:** Date range, location filter, trip type segmentation
6. **DAX Measures:** YoY growth, moving averages, cumulative totals

The Power BI model uses a star schema with a Date dimension table for time intelligence calculations and efficient querying.

Installation and Usage

Prerequisites

- Python 3.8 or higher
- Power BI Desktop (latest version)
- SQL database client (MySQL Workbench, DBeaver, or pgAdmin)
- Git for version control

Setup Instructions

1. Clone the repository:
git clone <https://github.com/ayush-9955/Uber-Rides-.git>
cd Uber-Rides-
2. Install Python dependencies:
pip install pandas numpy matplotlib seaborn scikit-learn ydata-profiling
3. Load datasets:
 - Open Excel files in the Excel folder for raw data inspection
 - Import CSV files into your SQL database using provided schema
4. Run SQL queries:
 - Execute queries from the SQL folder in your database client
 - Modify connection strings as needed for your environment
5. Execute Python notebooks:
 - Navigate to the Python folder
 - Open Jupyter notebooks: jupyter notebook
 - Run cells sequentially for EDA and modeling
6. Open Power BI dashboards:
 - Launch Power BI Desktop
 - Open .pbix files from the Power BI folder
 - Refresh data connections to your local database

- Interact with slicers and filters for dynamic analysis
-

Project Workflow

1. **Data Collection:** Obtain Uber trip dataset (2016) with ride details including timestamps, locations, distances, and durations
 2. **Data Cleaning:** Remove duplicates, handle missing values, standardize date formats using Excel and Python pandas
 3. **SQL Database Setup:** Create relational database schema, import cleaned data, establish indexes for query optimization
 4. **Exploratory Data Analysis:** Generate statistical summaries, identify outliers, visualize distributions and correlations
 5. **Feature Engineering:** Create derived features such as fare proxy, time-based categories, distance bins
 6. **Advanced Analytics:** Apply machine learning concepts, time-series forecasting, and fuzzy logic for demand prediction
 7. **Visualization:** Build interactive Power BI dashboards with DAX measures, slicers, and drill-through capabilities
 8. **Business Insights:** Extract actionable recommendations for pricing, resource allocation, and customer engagement
 9. **Documentation:** Prepare comprehensive README, code comments, and presentation materials
-

Business Applications

- **Operations Management:** Optimize driver allocation based on predicted demand patterns
 - **Pricing Strategy:** Implement surge pricing during identified peak hours and high-demand zones
 - **Marketing Campaigns:** Target promotions to specific customer segments based on ride behavior
 - **Service Quality:** Reduce cancellation rates by addressing issues in problematic locations
 - **Expansion Planning:** Identify underserved areas with growth potential using geographic analysis
 - **Financial Forecasting:** Project revenue trends using time-series models for budget planning
-

Future Enhancements

1. Integration of real-time API data for live dashboard updates
 2. Machine learning models for ride duration prediction using scikit-learn
 3. Advanced fuzzy logic systems for dynamic fare optimization
 4. Customer sentiment analysis from ride reviews using NLP
 5. Mobile app integration for driver performance tracking
 6. Cloud deployment using Azure or AWS for scalability
-

Learning Outcomes

This project demonstrates proficiency in:

- End-to-end data analytics pipeline implementation
 - SQL query optimization and window functions
 - Python data manipulation with pandas and numpy
 - Statistical analysis and hypothesis testing
 - Data visualization best practices in Power BI
 - Business intelligence reporting and stakeholder communication
 - Version control with Git and GitHub collaboration
 - Documentation and technical writing for portfolio presentation
-

Contact Information

Ayush Rajput
Data Analytics Student
Chandigarh University

- **GitHub:** <https://github.com/ayush-9955>
 - **Location:** Patna, Bihar, India
 - **Project Repository:** <https://github.com/ayush-9955/Uber-Rides-.git>
-

Acknowledgments

This project was developed as part of data analytics coursework and portfolio building for technical interview preparation. Datasets used are publicly available Uber trip records. Special thanks to the open-source community for tools like pandas, Power BI, and SQL databases.

License

This project is available for educational and portfolio purposes. Please provide attribution when using or referencing this work.

★ **Star this repository if you found it helpful!**

Built for Data Analytics Interviews and Portfolio Projects 🚀

Last Updated: March 9, 2026

Compatible with: Power BI Desktop, Python 3.8+, MySQL/PostgreSQL